A Pseudo-Spectral Solver for the Shallow Water Equations on the Sphere: A Manual for `psuedo_spectral.py`

June 20, 2026

Contents

1 Shallow Water Equations on the Sphere in Vorticity–Divergence Form

The classical shallow water equations (SWE) on a rotating sphere of radius a may be written for a horizontal velocity field $\boldsymbol{v} = (u, v)$ and free-surface height h as

$$\frac{\partial \boldsymbol{v}}{\partial t} + (\boldsymbol{v} \cdot \nabla)\boldsymbol{v} + f\,\hat{\boldsymbol{k}} \times \boldsymbol{v} = -g\nabla h, \tag{1}$$

$$\frac{\partial h}{\partial t} + \nabla \cdot (h\,\boldsymbol{v}) = 0, \tag{2}$$

where $f = 2\Omega \sin\phi$ is the Coriolis parameter, Ω is the planetary angular velocity, g is gravity, and ϕ is latitude.

For a pseudo-spectral discretization based on spherical harmonics it is convenient to replace the prognostic variables (u, v, h) by the vorticity, divergence, and geopotential

$$\zeta \;=\; \hat{\boldsymbol{k}} \cdot (\nabla \times \boldsymbol{v}), \qquad \delta \;=\; \nabla \cdot \boldsymbol{v}, \qquad \Phi \;=\; g\,h. \tag{3}$$

1.1 Helmholtz Decomposition on the Sphere

Any sufficiently smooth tangent vector field $\boldsymbol{v}$ on the two-sphere S^2 admits a unique decomposition into a *rotational* (divergence-free) part and an *irrotational* (curl-free) part:

$$\boldsymbol{v} \;=\; \underbrace{\hat{\boldsymbol{k}} \times \nabla\psi}_{\text{rotational}} + \underbrace{\nabla\chi}_{\text{irrotational}}, \tag{4}$$

where ψ is the *stream function* and χ is the *velocity potential*. The decomposition follows from the Hodge theorem on a compact orientable manifold: every 1-form on S^2 splits as $\omega = d\chi + \star d\psi + h$, where h is harmonic. Because S^2 has trivial first cohomology, the harmonic part vanishes, leaving only the two scalar potentials. Uniqueness of ψ, χ is enforced by requiring zero global mean, $\int_{S^2} \psi\, dS = \int_{S^2} \chi\, dS = 0$.

Taking the (scalar) curl and the divergence of (**??**) yields

$$\zeta \;\equiv\; \hat{\boldsymbol{k}}\cdot(\nabla\times\boldsymbol{v}) \;=\; \nabla^2\psi, \qquad \delta \;\equiv\; \nabla\cdot\boldsymbol{v} \;=\; \nabla^2\chi, \tag{5}$$

using the spherical identities $\nabla\cdot(\hat{\boldsymbol{k}}\times\nabla\psi)=0$ and $\hat{\boldsymbol{k}}\cdot(\nabla\times\nabla\chi)=0$. Thus knowing (ζ,δ) determines (ψ,χ) uniquely up to a constant by inverting the Laplace–Beltrami operator on S^2, and hence determines the velocity through (**??**). This is exactly the operation performed by the routine `getuv`.

1.2 Diagonalisation of the Laplacian in the Spherical Harmonic Basis

The Laplace–Beltrami operator on a sphere of radius a in (λ,ϕ) coordinates ($\lambda\in[0,2\pi)$ longitude, $\phi\in[-\pi/2,\pi/2]$ latitude) reads

$$\nabla^2 f \;=\; \frac{1}{a^2\cos\phi}\frac{\partial}{\partial\phi}\!\left(\cos\phi\,\frac{\partial f}{\partial\phi}\right) + \frac{1}{a^2\cos^2\phi}\,\frac{\partial^2 f}{\partial\lambda^2}. \tag{6}$$

The (real, normalized) spherical harmonics

$$Y_{\ell m}(\lambda,\phi) \;=\; N_{\ell m}\,P_\ell^{|m|}(\sin\phi)\begin{cases}\cos(m\lambda), & m\ge 0,\\ \sin(|m|\lambda), & m<0,\end{cases} \qquad \ell\ge 0,\ |m|\le\ell, \tag{7}$$

are eigenfunctions of ∇^2 with eigenvalues $-\ell(\ell+1)/a^2$.

Proof. Writing $\mu=\sin\phi$ so that $\cos\phi\,d\phi=d\mu$ and $\cos^2\phi=1-\mu^2$, (**??**) becomes

$$\nabla^2 f \;=\; \frac{1}{a^2}\left[\frac{\partial}{\partial\mu}\Big((1-\mu^2)\frac{\partial f}{\partial\mu}\Big) + \frac{1}{1-\mu^2}\,\frac{\partial^2 f}{\partial\lambda^2}\right]. \tag{8}$$

Substituting $Y_{\ell m}(\lambda,\mu)=P_\ell^{|m|}(\mu)\,e^{im\lambda}$ (complex form is sufficient for the eigenvalue calculation),

$$\frac{\partial^2 Y_{\ell m}}{\partial\lambda^2} = -m^2\,Y_{\ell m},$$
$$\frac{\partial}{\partial\mu}\Big((1-\mu^2)\frac{\partial Y_{\ell m}}{\partial\mu}\Big) = e^{im\lambda}\,\frac{d}{d\mu}\Big((1-\mu^2)\frac{dP_\ell^{|m|}}{d\mu}\Big).$$

The associated Legendre function $P_\ell^{|m|}$ satisfies the associated Legendre equation

$$\frac{d}{d\mu}\Big((1-\mu^2)\frac{dP_\ell^{|m|}}{d\mu}\Big) + \Big(\ell(\ell+1)-\frac{m^2}{1-\mu^2}\Big)P_\ell^{|m|} \;=\; 0, \tag{9}$$

hence

$$\frac{d}{d\mu}\Big((1-\mu^2)\frac{dP_\ell^{|m|}}{d\mu}\Big) \;=\; -\ell(\ell+1)P_\ell^{|m|} + \frac{m^2}{1-\mu^2}P_\ell^{|m|}. \tag{10}$$

Plugging back in,

$$\nabla^2 Y_{\ell m} \;=\; \frac{1}{a^2}\Big[-\ell(\ell+1)+\frac{m^2}{1-\mu^2}-\frac{m^2}{1-\mu^2}\Big]Y_{\ell m} \;=\; -\frac{\ell(\ell+1)}{a^2}\,Y_{\ell m}. \qquad \blacksquare$$

Because $\{Y_{\ell m}\}$ is a complete orthonormal basis of $L^2(S^2)$, an arbitrary field f expands as $f = \sum_{\ell,m} \hat{f}_{\ell m}\, Y_{\ell m}$, and the action of the Laplacian on the coefficient vector is the diagonal multiplication

$$\widehat{\nabla^2 f}_{\ell m} \;=\; -\frac{\ell(\ell+1)}{a^2}\,\hat{f}_{\ell m}, \qquad \widehat{\nabla^{-2} f}_{\ell m} \;=\; -\frac{a^2}{\ell(\ell+1)}\,\hat{f}_{\ell m} \quad (\ell \geq 1), \tag{11}$$

with the $\ell = 0$ mode (a global constant) lying in the kernel and treated separately. Two consequences are immediate:

- Solving the Poisson problems (**??**) for ψ, χ from (ζ, δ) reduces to one division per mode — the entire elliptic inversion is $O(N^2)$ once the transform itself has been computed.
- Higher powers ∇^{2k} used in hyperdiffusion are equally diagonal, with eigenvalues $\big(-\ell(\ell+1)/a^2\big)^k$.

From (ζ, δ) back to (u, v). On the sphere the horizontal gradient and the rotated gradient have clean expressions in vector spherical harmonics $\boldsymbol{\Psi}_{\ell m} = \nabla Y_{\ell m}$ and $\boldsymbol{\Phi}_{\ell m} = \hat{\boldsymbol{k}} \times \nabla Y_{\ell m}$:

$$\boldsymbol{v} \;=\; \sum_{\ell \geq 1, m} \Big[\hat{\psi}_{\ell m}\, \boldsymbol{\Phi}_{\ell m} + \hat{\chi}_{\ell m}\, \boldsymbol{\Psi}_{\ell m}\Big], \qquad \hat{\psi}_{\ell m} = -\frac{a^2}{\ell(\ell+1)}\hat{\zeta}_{\ell m}, \quad \hat{\chi}_{\ell m} = -\frac{a^2}{\ell(\ell+1)}\hat{\delta}_{\ell m}. \tag{12}$$

This is precisely what the (inverse) vector SHT does in `getuv`: multiply $(\hat{\zeta}, \hat{\delta})$ by the diagonal factor $-a^2/[\ell(\ell+1)]/a = -a/[\ell(\ell+1)]$ and apply the inverse vector transform to get (u, v). In the code, `self.lap * self.radius` carries the $\ell(\ell+1)/a$ factor (with a minus sign absorbed into `invlap`), exactly inverting this map in `vrtdivspec`.

Potential Vorticity

The shallow-water potential vorticity (PV) is the absolute vorticity divided by the layer thickness,

$$q \;=\; \frac{\zeta + f}{h}. \tag{13}$$

For an inviscid flow, q is materially conserved, $Dq/Dt = 0$, and is a central diagnostic of the Galewsky test case. In the code a non-dimensionalised version is computed,

$$\tilde{q} \;=\; \frac{\bar{h}\, g}{2\Omega}\,\frac{\zeta + f}{\Phi}, \tag{14}$$

where $\bar{h}$ is the mean layer depth.

Prognostic System in (Φ, ζ, δ)

Taking the curl and divergence of (**??**) and combining with (**??**) yields the equations actually integrated:

$$\frac{\partial \zeta}{\partial t} = -\nabla \cdot \big[(\zeta + f)\,\boldsymbol{v}\big], \tag{15}$$

$$\frac{\partial \delta}{\partial t} = \hat{\boldsymbol{k}} \cdot \nabla \times \big[(\zeta + f)\,\boldsymbol{v}\big] - \nabla^2\Big(\Phi + \tfrac{1}{2}|\boldsymbol{v}|^2\Big), \tag{16}$$

$$\frac{\partial \Phi}{\partial t} = -\nabla \cdot (\Phi\,\boldsymbol{v}). \tag{17}$$

In a vector spherical harmonic basis the operator $\nabla\cdot$ and $\hat{\boldsymbol{k}}\cdot\nabla\times$ acting on a horizontal vector field return, respectively, the divergence and vorticity spectral coefficients (up to a factor of the radius and the eigenvalue of the Laplacian); this is what the routine `vrtdivspec` performs.

Equations (**??**)–(**??**) together with the diagnostic relation $\boldsymbol{v} = \nabla^{-2}(\,\hat{\boldsymbol{k}}\times\nabla\zeta + \nabla\delta\,)$ form a closed system; this is the system implemented in `psuedo_spectral.py`.

2 Algorithm in Pseudocode

The solver advances the state vector $\boldsymbol{U} = (\hat{\Phi},\,\hat{\zeta},\,\hat{\delta})$ of spectral coefficients with a third-order Adams–Bashforth scheme. Nonlinear products are evaluated on the physical grid (the pseudo-spectral idea) while time stepping and the Laplacian-based diagnostics are performed in spectral space. Implicit hyperdiffusion is applied to ζ and δ at the end of each step.

Algorithm 1 Pseudo-spectral integration of the shallow-water equations.

Input: n_{lat}, n_{lon}, Δt, physical constants $(a, \Omega, g, \bar{h}, \hat{h})$, number of steps N.
Precompute: Gaussian / Clenshaw–Curtiss latitudes and weights; spherical-harmonic transforms (scalar and vector); Laplacian eigenvalues $\lambda_\ell = -\ell(\ell+1)/a^2$ and inverse λ_ℓ^{-1}; Coriolis field $f = 2\Omega\sin\phi$; hyperdiffusion operator $\mathcal{H} = \exp\!\big(-\frac{\Delta t}{2\cdot 3600}(\lambda/\lambda_{\max})^4\big)$.
Build initial condition $\boldsymbol{U}^0$ from the Galewsky jet: compute $u(\phi)$, optional perturbation h_{bump}, transform to $(\hat{\zeta},\hat{\delta})$ via `vrtdivspec`, and obtain $\hat{\Phi}$ from the nonlinear balance equation.
for $n = 0, 1, \ldots, N-1$ **do**
    $\mathrm{tend}^{(\mathrm{new})} \leftarrow$ `dudtspec`$(\boldsymbol{U}^n)$ ▷ Evaluate RHS (**??**)–(**??**)
    if $n = 0$ **then** $\mathrm{tend}^{(\mathrm{now})} \leftarrow \mathrm{tend}^{(\mathrm{old})} \leftarrow \mathrm{tend}^{(\mathrm{new})}$ ▷ Forward Euler bootstrap
    else if $n = 1$ **then** $\mathrm{tend}^{(\mathrm{old})} \leftarrow \mathrm{tend}^{(\mathrm{new})}$ ▷ 2nd-order AB bootstrap
    end if
    $\boldsymbol{U}^{n+1} \leftarrow \boldsymbol{U}^n + \Delta t\Big(\frac{23}{12}\mathrm{tend}^{(\mathrm{new})} - \frac{16}{12}\mathrm{tend}^{(\mathrm{now})} + \frac{5}{12}\mathrm{tend}^{(\mathrm{old})}\Big)$
    $(\hat{\zeta},\hat{\delta})^{n+1} \leftarrow \mathcal{H}\cdot(\hat{\zeta},\hat{\delta})^{n+1}$ ▷ Implicit hyperdiffusion
    Cyclically permute the tendency buffers $(\mathrm{new}, \mathrm{now}, \mathrm{old})$.
end for
Return $\boldsymbol{U}^N$.

The right-hand-side routine `dudtspec` implements equations (**??**)–(**??**) in the following pseudocode form:

Algorithm 2 `dudtspec`: tendency of $(\hat{\Phi},\hat{\zeta},\hat{\delta})$.

Transform spectral state to the grid: $(\Phi,\zeta,\delta) \leftarrow$ `spec2grid`$(\boldsymbol{U})$.
Recover wind from vorticity and divergence: $(u,v) \leftarrow$ `getuv`$(\hat{\zeta},\hat{\delta})$.
Form the flux $\boldsymbol{F} = (u,v)\,(\zeta + f)$.
$(\hat{A},\hat{B}) \leftarrow$ `vrtdivspec`$(\boldsymbol{F})$, set $\partial_t\hat{\delta} = \hat{A}$, $\partial_t\hat{\zeta} = -\hat{B}$.
Form the mass flux $\boldsymbol{G} = (u,v)\,\Phi$, compute $(\hat{C},\hat{D}) \leftarrow$ `vrtdivspec`$(\boldsymbol{G})$, set $\partial_t\hat{\Phi} = -\hat{D}$.
Add the gradient term: $\partial_t\hat{\delta} \mathrel{-}= \nabla^2\big[\Phi + \frac{1}{2}(u^2+v^2)\big]$.
Return $(\partial_t\hat{\Phi}, \partial_t\hat{\zeta}, \partial_t\hat{\delta})$.

3 Line-by-Line Translation of the Pseudocode to Python

In what follows each block of `psuedo_spectral.py` is shown together with the step of the algorithm it implements.

3.1 Module imports and device selection

The solver relies on `torch_harmonics` for the spherical-harmonic transforms and runs on GPU when available.

```
import torch
import torch.nn as nn
import numpy as np
from math import ceil
import os, sys

from torch_harmonics.sht import *
import torch_harmonics as harmonics
from torch_harmonics.quadrature import *
from torch_harmonics.quadrature import _precompute_longitudes, _precompute_latitudes

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
```

3.2 Constructor: `ShallowWaterSolver.__init__`

The constructor stores physical constants, builds the SHT objects, the Laplacian (and its inverse), the Coriolis field, and the hyperdiffusion operator described in the precompute step of Algorithm 1.

```
class ShallowWaterSolver(nn.Module):
    def __init__(self, nlat, nlon, dt, lmax=None, mmax=None,
                 grid="equiangular", radius=6.37122E6,
                 omega=7.292E-5, gravity=9.80616,
                 havg=10.e3, hamp=120.):
        super().__init__()
        # ---- store dt and grid sizes ----
        self.dt   = dt
        self.nlat = nlat
        self.nlon = nlon
        self.grid = grid

        # ---- physical constants (registered as buffers) ----
        self.register_buffer('radius',  torch.as_tensor(radius,  dtype=torch.float64))
        self.register_buffer('omega',   torch.as_tensor(omega,   dtype=torch.float64))
        self.register_buffer('gravity', torch.as_tensor(gravity, dtype=torch.float64))
        self.register_buffer('havg',    torch.as_tensor(havg,    dtype=torch.float64))
        self.register_buffer('hamp',    torch.as_tensor(hamp,    dtype=torch.float64))

        # ---- scalar and vector spherical harmonic transforms ----
        self.sht   = harmonics.RealSHT(nlat, nlon, lmax=lmax, mmax=mmax,
                                       grid=grid, csphase=False)
        self.isht  = harmonics.InverseRealSHT(nlat, nlon, lmax=lmax, mmax=mmax,
                                              grid=grid, csphase=False)
        self.vsht  = harmonics.RealVectorSHT(nlat, nlon, lmax=lmax, mmax=mmax,
                                             grid=grid, csphase=False)
        self.ivsht = harmonics.InverseRealVectorSHT(nlat, nlon, lmax=lmax, mmax=mmax,
                                                    grid=grid, csphase=False)
        self.lmax = self.sht.lmax
        self.mmax = self.sht.mmax
```

The quadrature points are computed in a grid-dependent way (Gauss–Legendre, Lobatto, or equiangular / Clenshaw–Curtiss):

```
        if   self.grid == "legendre-gauss":
            cost, quad_weights = harmonics.quadrature.legendre_gauss_weights(self.nlat
    , -1, 1)
        elif self.grid == "lobatto":
            cost, quad_weights = harmonics.quadrature.lobatto_weights(self.nlat, -1,
    1)
        elif self.grid == "equiangular":
            cost, quad_weights = harmonics.quadrature.clenshaw_curtiss_weights(self.
    nlat, -1, 1)
        quad_weights = quad_weights.reshape(-1, 1)
        lats = -torch.arcsin(cost)                      # latitude grid
        lons = _precompute_longitudes(self.nlon)        # longitude grid
```

The Laplacian eigenvalues $-\ell(\ell+1)/a^2$ and the inverse Laplacian (with $\ell = 0$ excluded) are built next, followed by the Coriolis field $2\Omega \sin\phi$ and the spectral hyperdiffusion operator $\mathcal{H}$:

```
        # ---- Laplacian and its inverse ----
        l    = torch.arange(0, self.lmax).reshape(self.lmax, 1).double()
        l    = l.expand(self.lmax, self.mmax)
        lap     = - l * (l + 1) / self.radius**2
        invlap  = - self.radius**2 / l / (l + 1)
        invlap[0] = 0.

        # ---- Coriolis force ----
        coriolis = 2 * self.omega * torch.sin(lats).reshape(self.nlat, 1)

        # ---- spectral hyperdiffusion ----
        hyperdiff = torch.exp(torch.asarray(
            (-self.dt / 2 / 3600.) * (lap / lap[-1, 0])**4))

        # register all precomputed tensors
        for name, val in [('lats',lats),('lons',lons),('l',l),
                          ('lap',lap),('invlap',invlap),
                          ('coriolis',coriolis),('hyperdiff',hyperdiff),
                          ('quad_weights',quad_weights)]:
            self.register_buffer(name, val)
```

3.3 Spectral / grid utilities

These wrap the scalar SHT to move between physical and spectral space.

```
    def grid2spec(self, ugrid):
        return self.sht(ugrid)

    def spec2grid(self, uspec):
        return self.isht(uspec)
```

3.4 Vorticity / divergence transforms

`vrtdivspec` converts a horizontal vector field on the grid to $(\hat{\zeta}, \hat{\delta})$ by applying the vector SHT and multiplying by $a\,\lambda_\ell$. `getuv` is its inverse and corresponds to the Helmholtz reconstruction $\boldsymbol{v} = \hat{\boldsymbol{k}} \times \nabla\psi + \nabla\chi$.

```
    def vrtdivspec(self, ugrid):
        # multiplying by lap * radius converts the vector-SHT coefficients
```

```
        # of (u,v) into spectral coefficients of (zeta, delta)
        return self.lap * self.radius * self.vsht(ugrid)

    def getuv(self, vrtdivspec):
        # inverse: divide by lap*radius and apply inverse vector SHT
        return self.ivsht(self.invlap * vrtdivspec / self.radius)
```

The convenience routine `gethuv` reconstructs (h, u, v) on the grid from the full state vector.

```
    def gethuv(self, uspec):
        hgrid  = self.spec2grid(uspec[:1])
        uvgrid = self.getuv(uspec[1:])
        return torch.cat((hgrid, uvgrid), dim=-3)
```

3.5 Diagnostics: potential vorticity and non-dimensionalisation

Equation (**??**) is computed directly on the grid:

```
    def potential_vorticity(self, uspec):
        ugrid = self.spec2grid(uspec)
        pvrt  = (0.5 * self.havg * self.gravity / self.omega) \
                * (ugrid[1] + self.coriolis) / ugrid[0]
        return pvrt
```

```
    def dimensionless(self, uspec):
        # geopotential perturbation relative to background
        uspec[0]  = (uspec[0] - self.havg * self.gravity) \
                    / self.hamp / self.gravity
        # vorticity/divergence scaled by sqrt(gH)/a
        uspec[1:] = uspec[1:] * self.radius / torch.sqrt(self.gravity * self.havg)
        return uspec
```

3.6 Tendency: `dudtspec` (Algorithm 2)

Each line below matches one step of Algorithm 2.

```
    def dudtspec(self, uspec):
        dudtspec = torch.zeros_like(uspec)

        # (line 1) spectral -> grid
        ugrid  = self.spec2grid(uspec)
        # (line 2) reconstruct (u,v) from (vrt, div)
        uvgrid = self.getuv(uspec[1:])

        # (line 3) flux  F = (u,v) * (zeta + f)
        tmp     = uvgrid * (ugrid[1] + self.coriolis)
        # (line 4) -> spectral (vrt, div) of F
        tmpspec = self.vrtdivspec(tmp)
        dudtspec[2] =  tmpspec[0]      # d_t div
        dudtspec[1] = -tmpspec[1]      # d_t vrt

        # (line 5) mass flux G = Phi * (u,v)
        tmp = uvgrid * ugrid[0]
        tmp = self.vrtdivspec(tmp)
        dudtspec[0] = -tmp[1]          # d_t Phi

        # (line 6) gradient term  - laplacian( Phi + 0.5 |v|^2 )
        tmpspec = self.grid2spec(ugrid[0] + 0.5 * (uvgrid[0]**2 + uvgrid[1]**2))
```

```
        dudtspec[2] = dudtspec[2] - self.lap * tmpspec

        return dudtspec
```

3.7 Initial condition: `galewsky_initial_condition`

Builds the analytic mid-latitude jet of Galewsky *et al.* (2004), adds the optional Gaussian height bump (and noise) and inverts the nonlinear balance equation in spectral space.

```
    def galewsky_initial_condition(self, umax=80., usouth=1/7, unorth=5/14,
                                   perturb_loc=0.25, perturb_amp=1.,
                                   noise_level=0):
        device = self.lap.device

        # (a) southern / northern edge of the jet and the normalisation
        phi0 = torch.asarray(torch.pi * usouth, device=device)
        phi1 = torch.asarray(torch.pi * unorth, device=device)
        phi2 = perturb_loc * torch.pi
        en   = torch.exp(torch.asarray(-4.0 / (phi1 - phi0)**2, device=device))
        alpha, beta = 1./3., 1./15.

        lats, lons = torch.meshgrid(self.lats, self.lons)

        # (b) zonal jet velocity u1 supported on (phi0, phi1)
        u1    = (umax/en) * torch.exp(1./((lats-phi0)*(lats-phi1)))
        ugrid = torch.where(torch.logical_and(lats < phi1, lats > phi0),
                            u1, torch.zeros(self.nlat, self.nlon, device=device))
        vgrid = torch.zeros((self.nlat, self.nlon), device=device)

        # (c) optional Gaussian noise + localised height bump
        if noise_level > 0:
            noise = noise_level * torch.randn(self.nlat, self.nlon, device=device)
        else:
            noise = torch.zeros(self.nlat, self.nlon, device=device)

        hbump = noise + self.hamp * perturb_amp \
                * torch.cos(lats) \
                * torch.exp(-((lons-torch.pi)/alpha)**2) \
                * torch.exp(-(phi2-lats)**2/beta)

        # (d) wind -> (vrt, div) spectral
        ugrid       = torch.stack((ugrid, vgrid))
        vrtdivspec  = self.vrtdivspec(ugrid)
        vrtdivgrid  = self.spec2grid(vrtdivspec)

        # (e) nonlinear balance to obtain geopotential
        tmp         = ugrid * (vrtdivgrid[:1] + self.coriolis)
        tmpspec     = self.vrtdivspec(tmp)
        tmpspec[1]  = self.grid2spec(0.5 * torch.sum(ugrid**2, dim=0))
        phispec     = self.invlap*tmpspec[0] - tmpspec[1] \
                      + self.grid2spec(self.gravity*(self.havg + hbump))

        # (f) assemble state vector [Phi, vrt, div] in spectral space
        uspec      = torch.zeros(3, self.lmax, self.mmax,
                                 dtype=vrtdivspec.dtype, device=device)
        uspec[0]   = phispec
        uspec[1:]  = vrtdivspec
        return torch.tril(uspec)
```

3.8 Time integration: `timestep` (Algorithm 1, loop body)

This routine performs the third-order Adams–Bashforth update with forward-Euler / AB2 bootstrap and the implicit hyperdiffusion step.

```
    def timestep(self, uspec, nsteps):
        # storage for tendencies at the three relevant time levels
        dudtspec = torch.zeros(3, 3, self.lmax, self.mmax,
                               dtype=uspec.dtype, device=uspec.device)
        inew, inow, iold = 0, 1, 2

        for iter in range(nsteps):
            # ---- evaluate RHS ----
            dudtspec[inew] = self.dudtspec(uspec)

            # ---- bootstrapping: forward Euler, then AB2, then AB3 ----
            if iter == 0:
                dudtspec[inow] = dudtspec[inew]
                dudtspec[iold] = dudtspec[inew]
            elif iter == 1:
                dudtspec[iold] = dudtspec[inew]

            # ---- AB3 update ----
            uspec = uspec + self.dt * (
                  (23./12.) * dudtspec[inew]
                - (16./12.) * dudtspec[inow]
                + ( 5./12.) * dudtspec[iold] )

            # ---- implicit hyperdiffusion on vrt, div ----
            uspec[1:] = self.hyperdiff * uspec[1:]

            # ---- cyclic permutation of time-level pointers ----
            inew = (inew - 1) % 3
            inow = (inow - 1) % 3
            iold = (iold - 1) % 3

        return uspec
```

3.9 Driver: `main`

The driver instantiates the solver, sweeps over an ensemble of Galewsky parameters, integrates for 30 days, and saves snapshots every 30 minutes. Only the essential lines are reproduced here.

```
def main():
    nlat = 120; nlon = 2*nlat
    lmax = ceil(64); mmax = lmax

    dt = 150                            # seconds
    save_interval = int(1800/dt)        # snapshot every 30 min
    nday = 15 * 2                       # 30 days

    swe_solver = ShallowWaterSolver(nlat, nlon, dt,
                                    lmax=lmax, mmax=mmax).to(device)

    cache_path = '/data/keeling/a/warrenh2/galewsky_simulations/' \
                 '120x240_galewsky_cache/'
    os.makedirs(cache_path, exist_ok=True)

```

```
    # Parameter sweep ranges (centred about the canonical Galewsky values)
    umax_list        = [80. + i1 * 20         for i1 in range(-3, 3)]
    usouth_list      = [1/7  + i2 * 1/36.     for i2 in range(-2, 3)]
    unorth_list      = [5/14 + i2 * 1/36.     for i2 in range(-2, 3)]
    perturb_loc_list = [0.25 + i3 * 1/18.     for i3 in range(-2, 4)]
    perturb_amp_list = [1.   + i4 * 0.5       for i4 in range(-2, 3)]
    noise_level_list = [0]            # noise disabled (prof. request)

    i1 = 0; umax = umax_list[i1]
    for i2, usouth in enumerate(usouth_list):
        unorth = unorth_list[i2]
        for i3, perturb_loc in enumerate(perturb_loc_list):
            for i4, perturb_amp in enumerate(perturb_amp_list):
                for i5, noise_level in enumerate(noise_level_list):
                    filename = f'{i1}{i2}{i3}{i4}{i5}.pkl'
                    filepath = os.path.join(cache_path, filename)
                    if os.path.exists(filepath):
                        continue

                    # ---- (1) initial condition ----
                    uspec0 = swe_solver.galewsky_initial_condition(
                        umax=umax, usouth=usouth, unorth=unorth,
                        perturb_loc=perturb_loc, perturb_amp=perturb_amp,
                        noise_level=noise_level)

                    # ---- (2) time integration & snapshotting ----
                    data_list = []
                    for i in range(24*nday + 1):
                        data_list.append(uspec0.cpu())
                        uspec0 = swe_solver.timestep(uspec0, save_interval)

                    # ---- (3) save trajectory ----
                    tensor_save = torch.stack(data_list)
                    torch.save(tensor_save, filepath)

if __name__ == "__main__":
    main()
```

Mapping back to the algorithm. The constructor performs the precompute step of Algorithm 1. `galewsky_initial_condition` builds $\boldsymbol{U}^0$. `dudtspec` is Algorithm 2, evaluated inside the time loop of `timestep`, which implements the AB3 update and hyperdiffusion of Algorithm 1. `main` simply iterates the solver over the ensemble of Galewsky parameters and writes the resulting trajectories to disk.

4 Contour Advective Semi-Lagrangian (CASL) Method

The pseudo-spectral solver of Section 2 evaluates Eulerian flux divergences on a fixed grid and relies on hyperdiffusion to damp the unresolved cascade of potential-vorticity variance. CASL takes the opposite stance: PV is represented by a collection of Lagrangian *contours* which are simply advected by the local wind. The inviscid PV conservation law $Dq/Dt = 0$ is then exact along trajectories, sharp PV fronts (such as the Galewsky jet's edge) survive indefinitely, and no explicit dissipation is needed.

4.1 The CASL state and the two-grid idea

CASL carries two complementary representations of the flow at all times:

- a set of *PV level sets*, $\mathcal{C} = \{C_1, \ldots, C_K\}$, where each contour C_k encloses the region $\{q > q_k\}$ for a prescribed PV value q_k (the contours discretise the range of q at spacing Δq);
- a coarse *inversion grid* on which (u, v, Φ) are reconstructed at each step by solving the PV inversion problem.

Between inversion calls the contours move purely Lagrangianly; between contour-advection calls the wind is held fixed. A periodic *contour surgery* step removes filaments thinner than a user-set threshold and reconnects pinched contours.

4.2 PV inversion on the sphere

Given the PV field q and the mean depth $\bar{h}$, the auxiliary quantities are reconstructed by solving, at each step,

$$(\zeta + f) \;=\; q\,h, \qquad \nabla^2\psi \;=\; \zeta, \qquad \nabla^2\chi \;=\; \delta, \qquad \boldsymbol{v} \;=\; \hat{\boldsymbol{k}}\times\nabla\psi + \nabla\chi, \tag{18}$$

together with a balance relation that closes the system (e.g. linear balance $\nabla^2\Phi = \nabla\cdot(f\boldsymbol{v})$, or nonlinear/quasi-geostrophic balance). Each Poisson solve in (**??**) is diagonal in spherical harmonics by (**??**), so we re-use the SHT machinery of Section 2 even though the time stepping has changed completely.

4.3 Pseudocode

Algorithm 3 CASL on the sphere for the inviscid shallow-water equations.

Input: initial PV field $q^0(\lambda, \phi)$, mean depth $\bar{h}$, Coriolis f, time step Δt, contour spacing Δq, surgery threshold $\ell_{\min}$, number of steps N.
Initialise contours. Choose PV levels $q_k = q_{\min} + k\,\Delta q$; extract isolines of q^0 at each q_k to obtain $\mathcal{C}^0 = \{C_1, \ldots, C_K\}$.
Precompute (as in Section 2): SHT and inverse SHT, Laplacian eigenvalues $-\ell(\ell+1)/a^2$, Coriolis field, balance operator.
for $n = 0, 1, \ldots, N-1$ **do**
    (a) Render contours $\mathcal{C}^n$ onto the inversion grid: $q^n(\lambda_i, \phi_j) = q_{\min} + \Delta q\,\#\{C_k : (\lambda_i, \phi_j) \in \mathrm{int}(C_k)\}$.
    (b) PV inversion. With h^n from the previous step (or $\bar{h}$ at $n = 0$), compute $\zeta^n = q^n h^n - f$; spectrally invert $\nabla^2\psi = \zeta^n$, $\nabla^2\chi = \delta^n$; update h^n from the chosen balance relation; reconstruct $\boldsymbol{v}^n = \hat{\boldsymbol{k}}\times\nabla\psi + \nabla\chi$ via the inverse vector SHT.
    (c) Advect contour nodes. For every node $\boldsymbol{x}_p \in \mathcal{C}^n$ on every contour, integrate $\dot{\boldsymbol{x}}_p = \boldsymbol{v}^n(\boldsymbol{x}_p)$ for a step Δt using a semi-Lagrangian / third-order Runge–Kutta scheme; interpolate $\boldsymbol{v}^n$ to $\boldsymbol{x}_p$ from the inversion grid.
    (d) Node redistribution. Insert new nodes along contour arcs whose curvature times local spacing exceeds a tolerance; remove nodes where the spacing is too small.
    (e) Contour surgery (every n_{surg} steps). Detect contour segments closer than $\ell_{\min}$ and reconnect them; delete closed loops with perimeter $< \ell_{\min}$.
    $\mathcal{C}^{n+1} \leftarrow$ the updated contour set.
end for
Return $\mathcal{C}^N$ and the corresponding gridded fields $(q^N, \zeta^N, \delta^N, \Phi^N, \boldsymbol{v}^N)$.

4.4 Inversion sub-routine

Algorithm 4 `pv_invert`: $(q, \bar{h}) \mapsto (\boldsymbol{v}, \Phi, \zeta, \delta)$.

1: Initialise $h \leftarrow \bar{h}$, $\delta \leftarrow 0$.
2: **repeat**
3: $\quad \zeta \leftarrow q\,h - f$ ▷ from $q = (\zeta + f)/h$
4: $\quad \hat{\zeta} \leftarrow \texttt{SHT}(\zeta)$, $\hat{\delta} \leftarrow \texttt{SHT}(\delta)$
5: $\quad \hat{\psi}_{\ell m} \leftarrow -a^2/[\ell(\ell+1)]\,\hat{\zeta}_{\ell m}$ ▷ $\ell = 0$ mode zeroed
6: $\quad \hat{\chi}_{\ell m} \leftarrow -a^2/[\ell(\ell+1)]\,\hat{\delta}_{\ell m}$
7: $\quad \boldsymbol{v} \leftarrow \texttt{iVSHT}(\hat{\zeta}, \hat{\delta})$ ▷ equivalent to `getuv`
8: $\quad$ Update Φ from balance, e.g. solve $\nabla^2\Phi = \nabla\cdot(f\boldsymbol{v}) - \nabla^2\big[\tfrac{1}{2}|\boldsymbol{v}|^2\big]$ spectrally (one diagonal divide).
9: $\quad h \leftarrow \Phi/g$
10: **until** $\|h^{(k+1)} - h^{(k)}\| < \varepsilon$
11: **Return** $(\boldsymbol{v}, \Phi, \zeta, \delta)$.

4.5 Where the SHT still pays off

Even though there is no explicit Eulerian flux divergence, hyperdiffusion, or grid-based PV tendency, every step of CASL contains an elliptic inversion (lines (b) of Algorithm 3 and the whole of Algorithm 4) of the form $\nabla^2\phi = s$. By (**??**) this becomes one multiplication per spectral mode, costing $O(N^2)$ once the transform itself has been applied. Concretely the SHT contributes:

1. Poisson solves for ψ, χ from (ζ, δ);
2. one extra diagonal divide for the balance equation that provides Φ (and hence h);
3. the vector inverse SHT that turns $(\hat{\zeta}, \hat{\delta})$ directly into (u, v) via vector spherical harmonics — the same operation used by `getuv` in Section 3;
4. natural handling of the pole: SHT eliminates the $1/\cos\phi$ singularity that haunts lat-lon finite-difference or multigrid Poisson solvers.

What CASL removes is exactly what was *not* diagonal in the SH basis to begin with: the nonlinear advection of q (now done by following contours) and the dissipation needed to keep that advection stable (no longer needed because contour surgery, not diffusion, controls the cascade). The SHT is therefore narrowly but firmly useful: it is the workhorse of the elliptic inversion step — the one linear operation in CASL that genuinely benefits from spectral diagonalisation.